SMART EVENT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

HARISH GAUTHAM S

(Roll No. 241401027)

in partial fulfilment of the End Semester Examination for the course

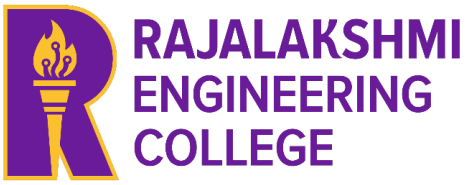
CB23F35

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND BUSINESS SYSTEMS



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu - 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled “Smart Event Management System” is the bonafide work of HARISH GAUTHAM S (Roll No. 241401027), submitted as an individual project for the course CB23F35 - Low-Code Application Development using Zoho Creator, during the academic year 2026-2027, Semester V.

The report presents the application design, demonstrated interfaces, system requirements, evaluation and proposed enhancements for an event management application developed on the Zoho Creator platform.

SIGNATURE

Dr. K.R. Sowmia
HEAD OF THE DEPARTMENT
Computer Science and Business Systems
Rajalakshmi Engineering College

SIGNATURE

Mr. M.L. Alphin Ezhil Manuel
COURSE FACULTY
Assistant Professor (SG)
Computer Science and Business Systems

Submitted for the End Semester Examination (Individual Project) held in November 2026.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

I, HARISH GAUTHAM S (Roll No. 241401027), submit this report titled “Smart Event Management System” for the course CB23F35 - Low-Code Application Development using Zoho Creator. The report describes my event management project, its user interfaces, data requirements and the scope for further development.

The application screenshots included in this report are from my Event Management System. The supplied reference report has been used only as a guide to academic structure and presentation; its visitor management application, code and test results are not presented as part of this project.

Features visible in the screenshots are distinguished from design recommendations and future enhancements. The illustrative Deluge examples in Appendix C are reference implementations for the proposed field names and are not represented as an export of the running application. External platform documentation is acknowledged in the references.

AI-assisted drafting and formatting were used in preparing this report. Responsibility for reviewing the report and confirming its consistency with the final application remains with me.

Place: Chennai **SIGNATURE OF THE STUDENT**

Date: 03.11.2026 HARISH GAUTHAM S

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Mr. S. Meganathan, B.E., F.I.E., Chairperson Dr. Thangam Meganathan, Ph.D., and Vice-Chairman Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S.N. Murugesan, M.E., Ph.D., Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. K.R. Sowmia, Head of the Department, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mr. M.L. Alphin Ezhil Manuel, Assistant Professor (SG), Department of Computer Science and Business Systems, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

HARISH GAUTHAM S

ABSTRACT

Event organisers need to coordinate registrations, venues, session schedules, expenses, approvals and participant communication. When these details are maintained in separate spreadsheets and messages, it becomes difficult to obtain a consistent view of event activity. The Smart Event Management System brings these functions into one Zoho Creator application with a common navigation menu and a dashboard. The demonstrated application contains menu sections for Events, Event Registrations, Venue Bookings, Expense Records, Approval Workflows, Session Agendas, Notification Logs and Analytics Summaries. The notification form captures the event, related registration, recipient details, message, communication channel and delivery status. The expense report provides a view of categories, estimated and actual costs, vendors and approval status. Analytics summaries display registrations, attendees, attendance rate, revenue, expenses, net profit and waitlist counts. The dashboard combines charts, delivery status counters and quick links, while a mobile preview demonstrates the notification form in a narrow layout. This report explains the requirements, proposed relational design, visible implementation and validation approach. A review of the five supplied screenshots confirms the interfaces and stored sample values but does not establish that automated emails, approval routing, access restrictions or production deployment have been tested. The report therefore separates demonstrated outputs from proposed business rules. The resulting prototype provides a foundation for centralised event coordination, with further work needed on event lookup labels, financial reconciliation, workflow verification and user acceptance testing.

Keywords: Smart Event Management; Low-Code Development; Zoho Creator; Event Registration; Expense Tracking; Dashboard; Notification Logs.

PROJECT AT A GLANCE

Project summary

Attribute	Details
Project title	Smart Event Management System
Student	Harish Gautham S
Roll number	241401027
Course	CB23F35 - Low-Code Application Development using Zoho Creator
Application display name	Event Management System
Application link name	event-management-system
Platform	Zoho Creator; trial banner visible in screenshots
Application domain	Event planning, coordination and reporting
Visible modules	Eight module sections plus Dashboard
Demonstrated outputs	Dashboard, notification form, expense report, analytics report and mobile preview
Visible sample records	3 expense rows and 5 analytics summary rows
Access	Owner session labelled Harish; independent role sessions not supplied
Automation and source	Workflow configuration and application source not supplied
Deployment	Browser access and mobile preview shown; environment not established
Evidence date	02 November 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS x
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Justification for a Low Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Existing System and 2.2 Limitations 3
2.3 Development Approaches and 2.4 Literature Survey 4
2.5 Emerging Directions and 2.6 Proposed System 5
REQUIREMENT ANALYSIS 6
3.1 Stakeholders and 3.2 Functional Requirements 6
3.3 Nonfunctional and 3.4 System Requirements 7
3.5 Use Case Model 8
SYSTEM DESIGN 9
4.1 System Architecture 9
4.2 Data Model and 4.3 Form Design 10
4.4 Process and Workflow Design 12
4.5 Navigation and 4.6 Access Control 13
IMPLEMENTATION 14
5.1 Application Setup and 5.2 Forms 14
5.3 Reports and 5.4 Dashboard 16
5.5 Deluge Scripting 17
5.6 Workflows and Automation 18

TABLE OF CONTENTS CONTINUED

5.7 Integrations 19
5.8 User Management and Role Based Access 20
5.9 AI and Analytics Features 21
5.10 Pro Code Extension and 5.11 Mobile Access 21
TESTING AND DEBUGGING 22
6.1 Testing Strategy and 6.2 Test Cases 22
6.3 Debugging Deluge Scripts 23
6.4 Defect Log and 6.5 User Acceptance Testing 24
DEPLOYMENT SECURITY AND GOVERNANCE 25
7.1 Deployment Strategy and 7.2 Performance 25
7.3 Security and 7.4 Governance 26
RESULTS AND DISCUSSION 27
8.1 Application Output 27
8.2 Achievement of Objectives 28
8.3 Comparison with the Existing System 29
8.4 Limitations and 8.5 Development Spectrum 29
CONCLUSION AND FUTURE ENHANCEMENTS 30
REFERENCES 31
APPENDIX A FEATURE COVERAGE CHECKLIST 32
APPENDIX B COURSE OUTCOME MAPPING 34
APPENDIX C ILLUSTRATIVE DELUGE SOURCE CODE 35
APPENDIX D USER MANUAL 39

LIST OF TABLES

Table 2.1 Comparison of development approaches 4
Table 2.2 Summary of platform literature 4
Table 3.1 Stakeholders and user roles 6
Table 3.2 Functional requirements 6
Table 3.3 Nonfunctional requirements 7
Table 3.4 Hardware and software requirements 7
Table 3.5 Use case description for expense approval 8
Table 4.1 Entities and relationships 10
Table 4.2 Field specification for Events 10
Table 4.3 Field specification for Event Registrations 11
Table 4.4 Field specification for Notification Logs 11
Table 4.5 Role permission matrix 13
Table 5.1 Reports demonstrated 16
Table 5.2 Proposed Deluge scripts 17
Table 5.3 Proposed workflows 18
Table 5.4 Integration scope 19
Table 6.1 Functional test cases 22
Table 6.2 Defect and observation log 24
Table 6.3 User acceptance plan 24
Table 7.1 Deployment checklist 25
Table 7.2 Performance evaluation plan 26
Table 7.3 Recommended security controls 26
Table 8.1 Mapping of objectives to outcomes 28
Table 8.2 Existing process versus proposed system 29
Table A.1 Feature coverage checklist 32
Table B.1 Course outcome mapping 34

LIST OF FIGURES

Figure 2.1 Existing event coordination process 3
Figure 3.1 Use case diagram 8
Figure 4.1 System architecture 9
Figure 4.2 Entity relationship diagram 10
Figure 4.3 Event registration and communication workflow 12
Figure 4.4 Dashboard navigation and wireframe 13
Figure 5.1 Application home and navigation 14
Figure 5.2 Notification form design schematic 15
Figure 5.3 Notification Logs desktop form 15
Figure 5.4 All Expense Records report 16
Figure 5.5 Dashboard page 17
Figure 5.6 Proposed workflow sequence 19
Figure 5.7 Proposed notification dispatch and logging 20
Figure 5.8 Recommended user access profiles 21
Figure 5.9 Notification Logs mobile preview 21
Figure 6.1 Workflow debugging and retest procedure 23
Figure 7.1 Proposed deployment stages 25
Figure 7.2 Proposed audit record structure 26
Figure 8.1 Notification entry interface 27
Figure 8.2 Expense approval status report 28
Figure 8.3 Event analytics summary report 28

Design diagrams describe the recommended event management structure. Supplied application screenshots are identified in their captions; diagrams do not represent unverified configuration screens.

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CO	Course Outcome
CRUD	Create Read Update Delete
CSV	Comma-Separated Values
Deluge	Data Enriched Language for the Universal Grid Environment
ER	Entity Relationship
HTTPS	Hypertext Transfer Protocol Secure
ID	Identifier
KPI	Key Performance Indicator
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator

Reading the implementation evidence

"Observed" refers to a feature or value visible in the supplied application screenshots. "Proposed" refers to a design or rule recommended for the system. "Not verified" means that source code, configuration or an executed test was not available to establish the behaviour. These distinctions apply throughout the report and prevent interface presence from being mistaken for a completed functional test.

CHAPTER 1
INTRODUCTION

1.1 Background

College programmes, technical workshops, conferences and cultural events involve multiple activities that must remain coordinated. Organisers maintain event details, allocate venues, accept registrations, prepare session agendas and control expenditure. Participants also need timely information about schedules and changes. A shared application can bring these records together and make the current status easier to review.

The Smart Event Management System uses Zoho Creator to organise event operations through forms, reports and a dashboard. The application name visible in the interface is Event Management System. This report uses Smart Event Management System as the project title; both names refer to the same project.

1.2 Problem Statement

A process based on disconnected spreadsheets and messages has no dependable common reference for an event. A registration may be counted in one file while an expense is recorded elsewhere. Approvals can become difficult to trace, and organisers may need to manually prepare attendance and financial summaries. The project addresses this fragmentation by organising related records around an event.

1.3 Objectives

• Provide a common workspace for event details, venue bookings, registrations and agendas.

• Capture communication details in a structured notification log.

• Track expenses and expose their approval status in reports.

• Present attendance and financial summaries through a dashboard.

• Support desktop use and review form usability in the mobile layout.

• Define validation, access control and test requirements for a dependable final system.

1.4 Scope of the Project

The application covers eight visible modules: Events, Event Registrations, Venue Bookings, Expense Records, Approval Workflows, Session Agendas, Notification Logs and Analytics Summaries. The supplied evidence demonstrates the notification form, two list reports, a dashboard and a mobile preview. The remaining modules are visible as navigation entries; their detailed field configurations are outside the supplied evidence.

The intended users are event administrators, organisers, finance or approval staff and participants. These are design roles. Only the owner session labelled Harish is visible, so configured permissions for separate users cannot be inferred.

1.5 Justification for a Low Code Approach

The project is primarily a structured information management problem. Event-related records can be represented using forms and viewed through reports. A visual development platform is suitable for quickly assembling these interfaces, while business rules can be expressed through workflow configuration and Deluge where necessary. Zoho describes forms, reports, workflows and pages as application components [1].

The practical benefit is that effort can focus on field definitions, relationships and user tasks. Low-code development still requires careful design: a convenient form does not ensure valid dates, a correct event association or secure access. The proposed implementation therefore includes validation rules and a test plan.

1.6 Organisation of the Report

Chapter 2 reviews the existing process and relevant platform documentation. Chapter 3 specifies requirements. Chapter 4 presents the system design. Chapter 5 documents the demonstrated implementation. Chapter 6 separates screenshot review from functional testing. Chapter 7 covers deployment, security and governance. Chapter 8 evaluates outputs, and Chapter 9 concludes with future enhancements. The appendices provide feature coverage, course mapping, illustrative scripts and a user manual.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The baseline process uses separate documents for event planning, registrations, bookings, expenses and participant communication. An organiser collects entries, contacts coordinators and prepares a final summary manually. Figure 2.1 models this process and the points where information can become inconsistent. It represents a typical manual workflow rather than a measured study of a particular institution.

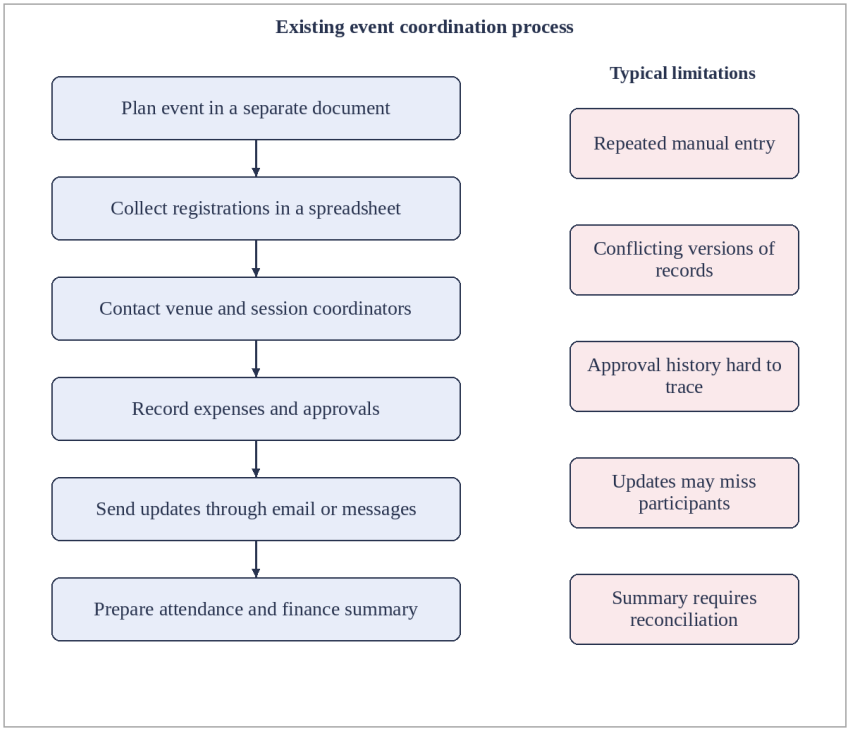


Figure 2.1: Existing event coordination process

2.2 Limitations of the Existing System

• Duplicate entry and inconsistent event names make records difficult to reconcile.

• Approval decisions and participant updates are scattered across messages.

• Attendance and financial summaries require manual preparation.

2.3 Low Code versus Traditional Development

Table 2.1: Comparison of development approaches

Criterion	Traditional development	Zoho Creator approach
Interface	Custom front end and endpoints	Visual forms and reports
Data	Separate schema and query layer	Forms and lookup relationships
Rules	Service or application code	Workflow rules and Deluge
Hosting	Managed separately	Platform-hosted application
Changes	Code, test and release	Edit configuration and validate
Customisation	Broad implementation control	Within supported platform features

The event system mainly requires structured data entry, record relationships and reporting. Low-code development suits these activities, while data validation and permissions remain development responsibilities.

2.4 Literature Survey

Table 2.2: Summary of platform literature

Source	Key concept	Project relevance
Zoho Creator applications [1]	Forms, reports, pages and workflows	Organise event operations
Form workflows [2]	Actions triggered by form events	Validate records and update state
Permissions [3]	Module and data-operation permissions	Separate organiser and finance work
Deluge sendmail [4]	Compose and send event-driven email	Plan dispatch independently of logging

2.4 Literature Survey Continued

The reviewed sources explain platform capabilities and how application components relate. They support the choice of forms for data capture, reports for record review and workflows for event-driven processing. They do not prove that any particular rule or integration has been configured in this project. Implementation claims therefore remain tied to the supplied application evidence.

2.5 Emerging Directions

Event systems can be extended with mobile check-in, generated QR passes, calendar integration and scheduled reminders. AI assistance can support application drafting or message preparation, but the screenshots do not establish that an AI feature was used. These possibilities are treated as future work rather than current functionality.

2.6 Proposed System

The proposed system uses an event record as the common reference for registrations, bookings, sessions, expenses and notifications. Organisers maintain operational details, authorised reviewers decide expenses, and the dashboard presents summary information. Each summary should be traceable to its underlying event records so that attendance and financial totals can be checked.

The demonstrated interface already brings these areas into one menu. The next design priority is consistency: event lookups should show meaningful event names, numeric summaries should use a defined calculation method, and notification status should reflect a verifiable communication outcome.

2.7 Design Implications

Each operational record should reference an event through a stable identifier. Reports should expose meaningful event names, and financial summaries should use a consistent currency and period. A notification record should retain the relationship between event, participant and message so that organisers can investigate communication failures.

The application can be developed incrementally: establish reliable fields and relationships first, then add validation and permission rules, and finally introduce integrations and automated reminders after controlled tests.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and user roles

Stakeholder	Responsibility
Administrator	Manage application configuration and user access
Organiser	Manage assigned events, registrations, venues and sessions
Finance reviewer	Review expenses and record approval decisions
Participant	Register and receive permitted event information

3.2 Functional Requirements

Table 3.2: Functional requirements

ID	Requirement	Role
FR01	Maintain event details and status	Organiser
FR02	Register participants against an event	Organiser / participant
FR03	Manage venue bookings and conflicts	Organiser
FR04	Maintain session agendas and times	Organiser
FR05	Capture and review event expenses	Organiser / reviewer
FR06	Capture notification recipient and content	Organiser
FR07	Calculate attendance and finance summaries	System / organiser
FR08	Display dashboard charts and shortcuts	Authorised users
FR09	Support usable mobile data entry	Authorised users
FR10	Restrict operations and retain change history	Administrator

These requirements describe the target system. The current screenshots demonstrate interfaces and sample values; workflow execution and separate role accounts require functional verification.

3.3 Nonfunctional Requirements

Table 3.3: Nonfunctional requirements

Requirement	Acceptance condition
Usability	Consistent labels; clear forms and validation feedback
Integrity	Valid event links, dates, costs and unique registrations
Security	Users can perform only their permitted operations
Traceability	Important decisions retain actor, time and reason
Reliability	Notification failures are recorded and safe to retry
Performance	Measure response with representative record volumes
Maintainability	Document field names, formulas and workflow dependencies

3.4 Hardware and Software Requirements

Table 3.4: Hardware and software requirements

Component	Requirement
Client	Desktop or laptop with a current web browser
Connectivity	Internet access to the Zoho Creator application
Account	Zoho account authorised for the application
Mobile review	Phone or Creator mobile preview; physical device for UAT
Development platform	Zoho Creator visual builders and Deluge where required
Test data	Controlled event, registration, expense and notification records

No dedicated local server or specialist hardware is evidenced. The observed application is accessed through a browser. The mobile screenshot is a layout preview rather than a physical-device test.

3.5 Use Case Model

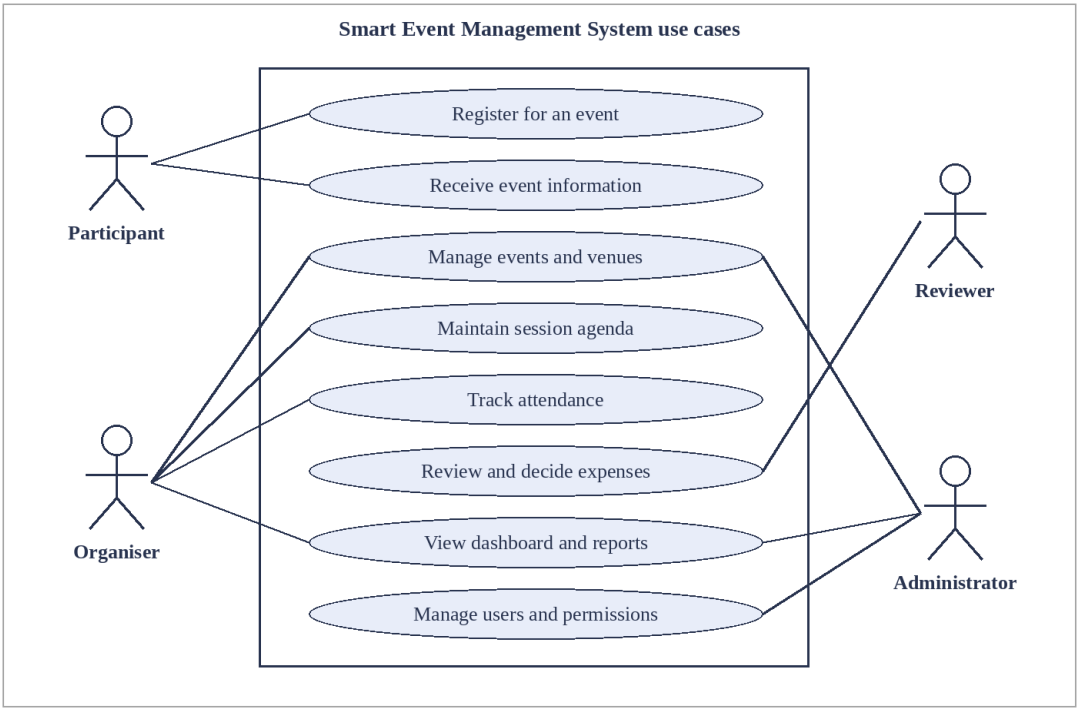


Figure 3.1: Use case diagram for the proposed system

Table 3.5: Use case description for expense approval

Attribute	Description
Use case ID	UC01
Actors	Finance reviewer; administrator where authorised
Precondition	Expense exists, is Pending, and is within reviewer scope
Main flow	Open expense; check event, cost and receipt; choose approve or reject; save decision with actor and time
Alternate flow	Missing receipt or unauthorised actor prevents a valid decision
Postcondition	Decision is stored and the expense report reflects its status
Related requirements	FR05 and FR10

The diagram and use case are design specifications. The supplied expense report shows status values but does not show the approval action or its enforcement.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

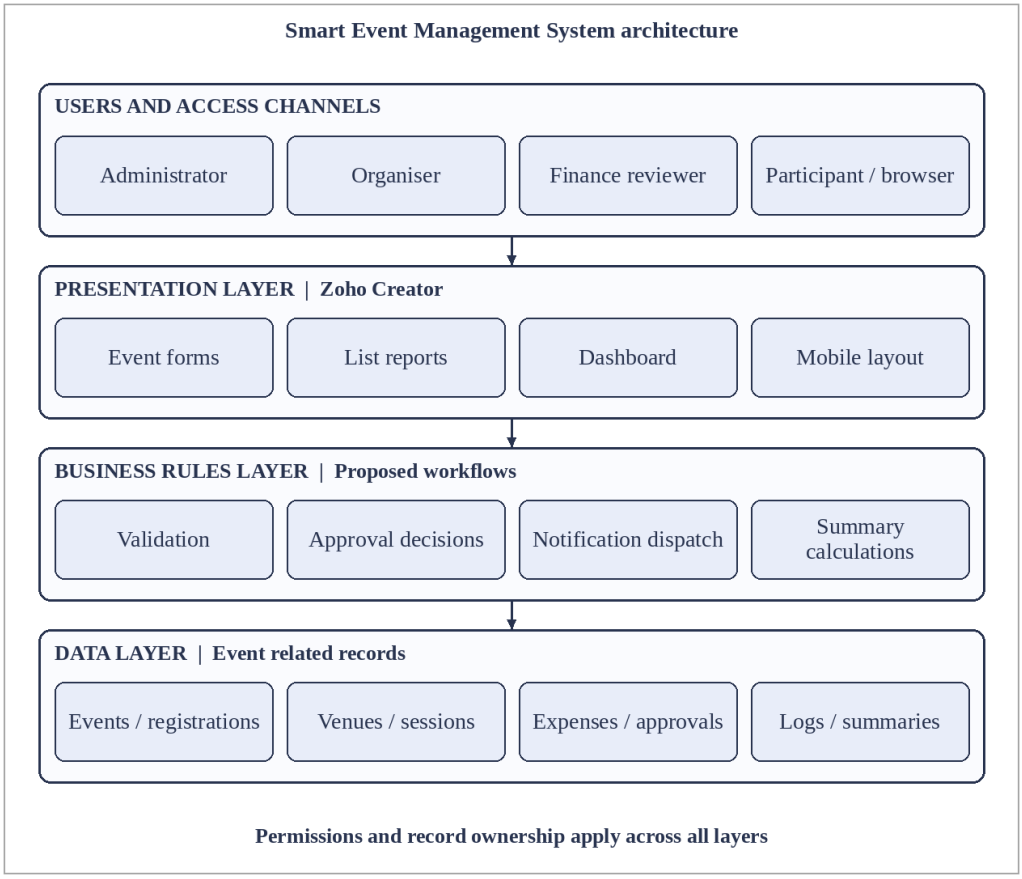


Figure 4.1: System architecture of Smart Event Management System

The presentation layer contains forms, reports, the dashboard and mobile layouts. The proposed business-rules layer validates records, handles approval decisions, dispatches notifications and refreshes summaries. The data layer stores events and their associated operational records. Permissions and record ownership apply across these layers.

The screenshots establish the presentation layer and stored sample outputs. Business rules and integration paths in the diagram are the intended design and require configuration review and testing.

4.2 Data Model

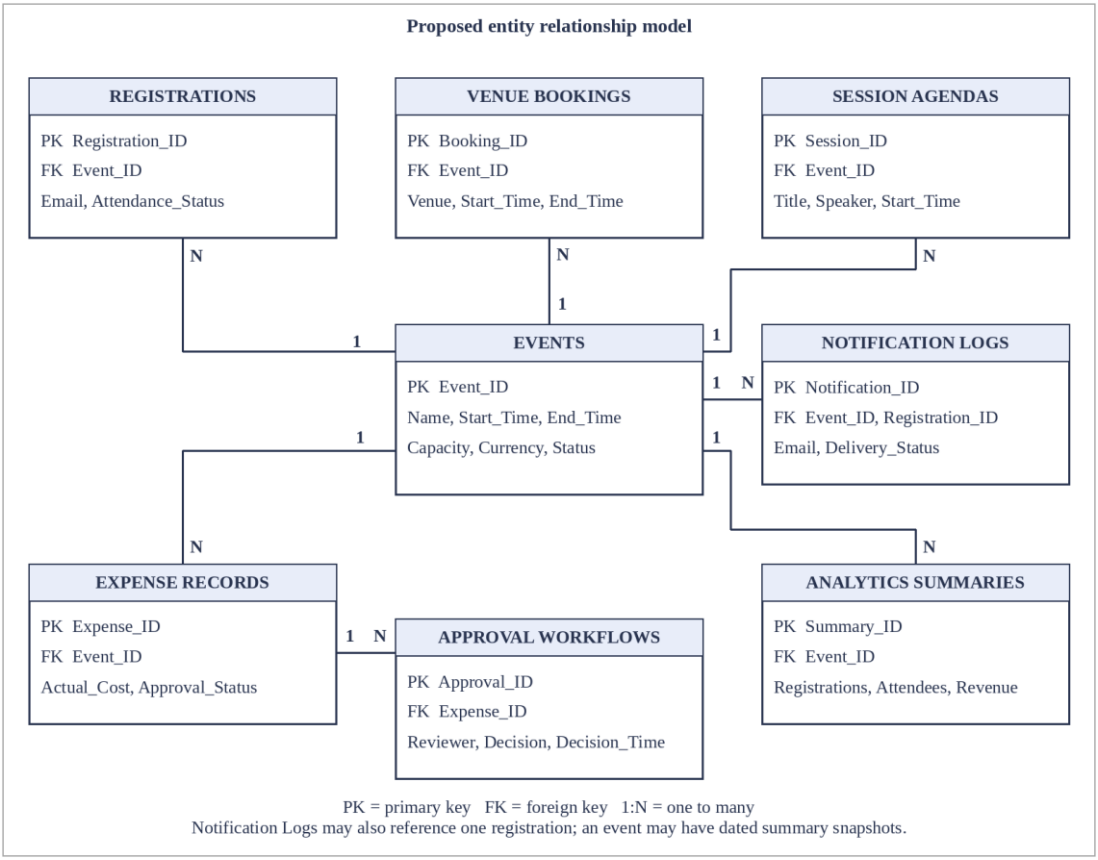


Figure 4.2: Proposed entity relationship diagram

Table 4.1: Entities and relationships

Parent	Related entity	Relationship
Events	Registrations, bookings and sessions	One to many
Events	Expenses, notification logs and summaries	One to many
Expense Records	Approval Workflows	One to many decisions
Event Registrations	Notification Logs	One to many; optional log link

4.3 Form Design

Table 4.2: Field specification for Events

Field	Type	Rule
Event_ID / Event_Name	ID / single line	Unique identity and readable display name
Start_Time / End_Time	Date-time	End must follow start
Capacity / Currency / Status	Number / choices	Positive capacity; explicit currency and lifecycle

4.3 Form Design Continued

Table 4.3: Field specification for Event Registrations

Proposed field	Type	Validation or purpose
Registration_ID	Record ID	Unique registration identity
Event_ID	Lookup	Must reference an existing event
Participant_Name	Name	Required participant identity
Participant_Email	Email	Valid address; check event/email duplicate
Registration_Time	Date-time	Set when registration is stored
Attendance_Status	Choice	Registered, Attended, Cancelled or Waitlisted

Table 4.4: Field specification for Notification Logs

Visible label	Control	Purpose
Event	Selection	Related event
Event Registrations	Selection	Related participant registration
Notification Type	Dropdown	Classify message
Recipient Email	Entry	Destination email
Recipient Name	Name	Recipient identity
Sent Time	Date-time	Dispatch timestamp
Delivery Status	Dropdown	Recorded communication state
Message Content	Multiline	Notification text
Channel	Radio buttons	Email, InApp or Other

Notification labels are taken from the supplied form. Registration field names and validation rules are proposed. Exact backend link names must be checked before deploying the example scripts in Appendix C.

Additional field design for operational records

Module	Key data	Recommended rule
Venue Bookings	Event, venue, start and end	Avoid confirmed overlapping bookings
Session Agendas	Event, session, speaker and time	Keep sessions inside event period
Expense Records	Event, cost, category, vendor and receipt	Nonnegative amounts; traceable approval
Analytics Summaries	Counts, rates, revenue and expenses	Use one event, period and currency

4.4 Process and Workflow Design

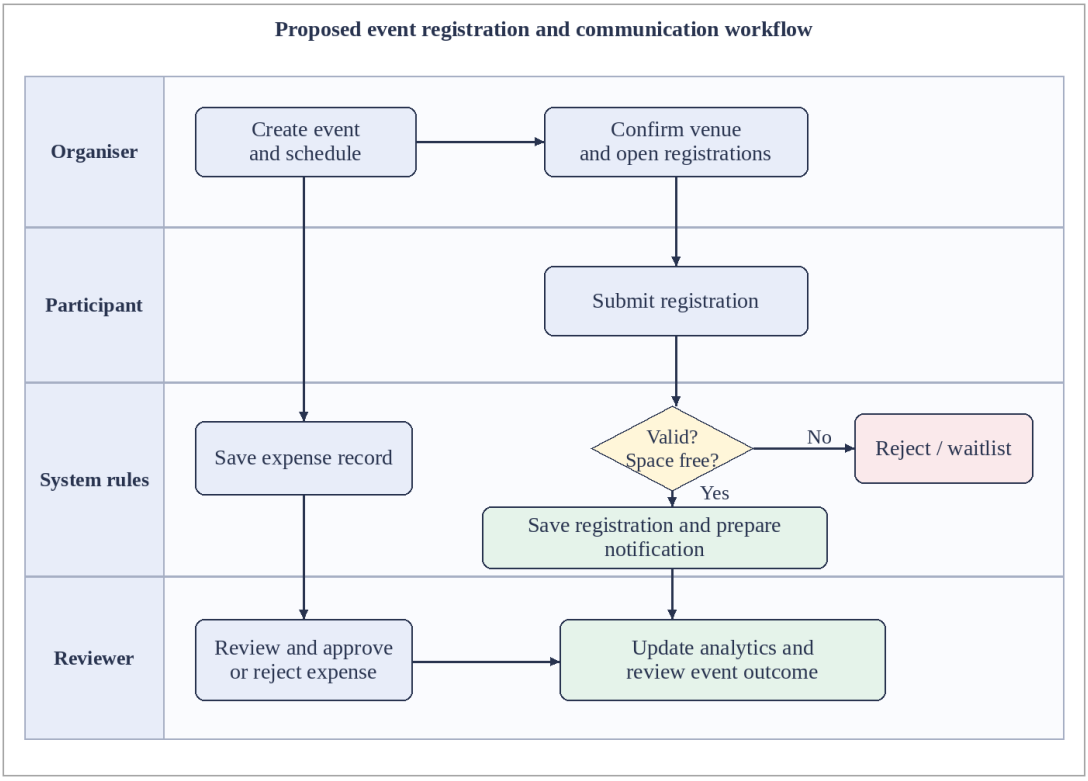


Figure 4.3: Proposed event registration and communication workflow

The flow combines event setup, registration validation, expense review and summary updates. Capacity checks and notification sending need runtime tests; they are not established by the navigation menu alone.

4.5 User Interface and Navigation Design

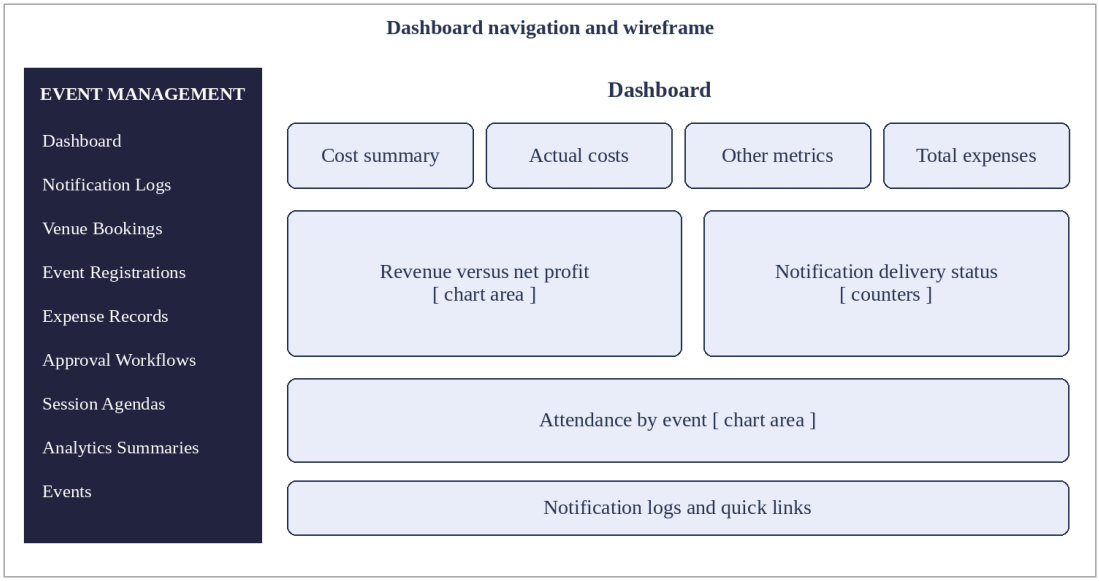


Figure 4.4: Dashboard navigation and wireframe

4.6 Access Control Design

Table 4.5: Recommended role permission matrix

Component	Admin	Organiser	Reviewer	Participant
Events and venues	Manage	Assigned	Read if needed	Published details
Registrations	Manage	Assigned	No default access	Own only
Expenses	Manage	Create / view	Review / decide	No access
Notification logs	Manage	Assigned	Relevant only	No shared log
Dashboard	All scope	Assigned scope	Finance scope	Own information
User settings	Manage	No access	No access	No access

The matrix defines intended permissions, not verified profiles. Separate accounts must be used to check direct links, exports and actions as well as visible menu entries. Hiding a menu does not secure the underlying records [3].

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application is displayed as Event Management System. The owner session is labelled Harish, and the sidebar contains Notification Logs, Venue Bookings, Event Registrations, Expense Records, Approval Workflows, Session Agendas, Analytics Summaries and Events.

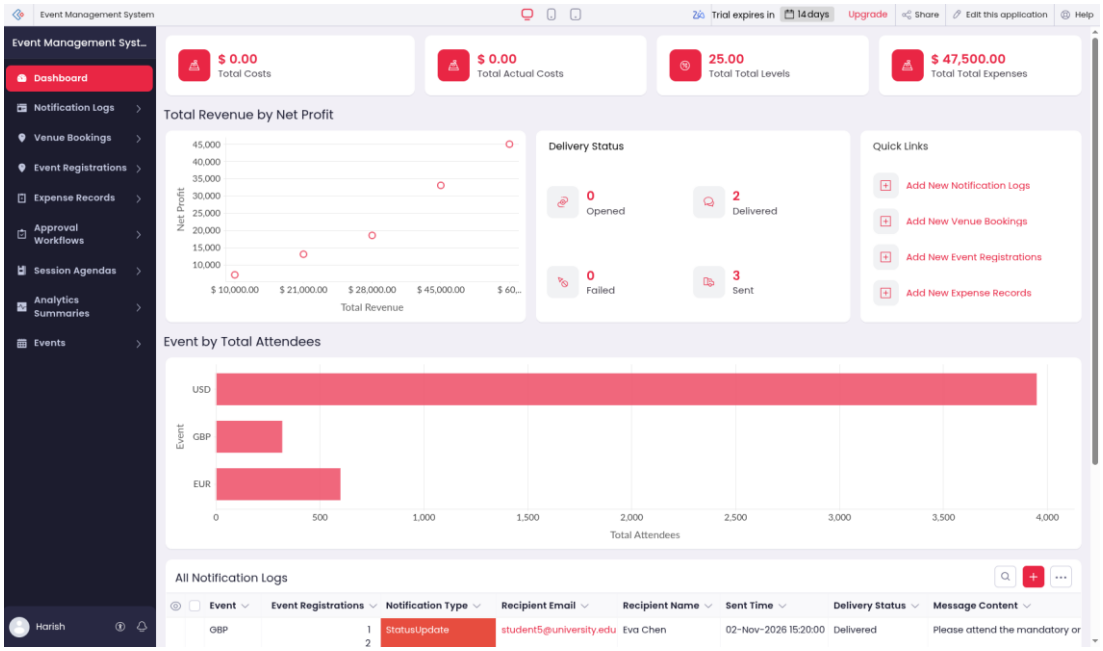


Figure 5.1: Application home and navigation sections

5.2 Forms and Data Capture

The notification form demonstrates structured data capture with event and registration selectors, recipient fields, message content and status information. The project uses the interface name Event Management System and the report title Smart Event Management System for the same application.

Only supplied screenshots are treated as implementation evidence. Design drawings used elsewhere explain the event-specific architecture without representing unverified application settings.

5.2 Forms and Data Capture Continued

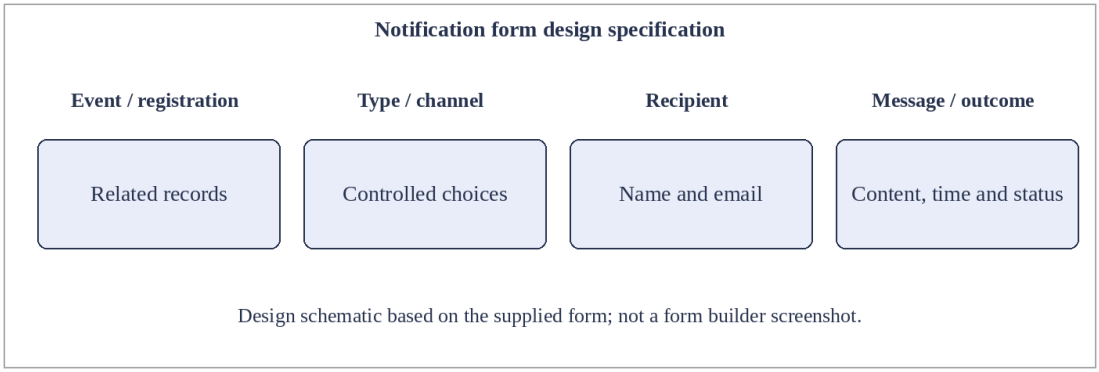


Figure 5.2: Notification form design schematic

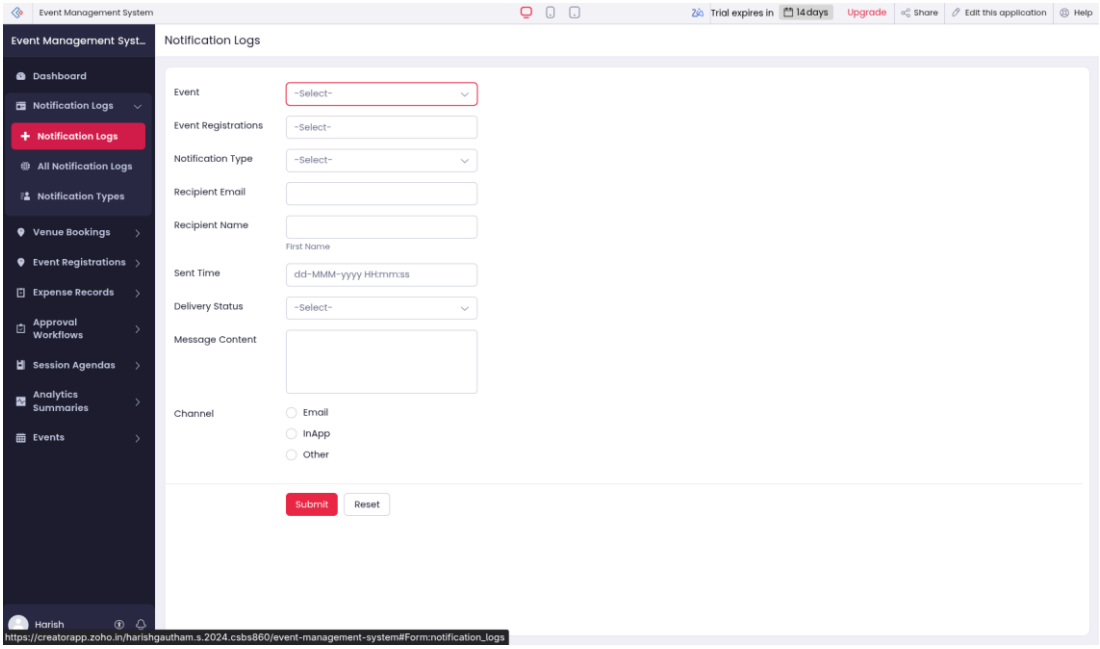


Figure 5.3: Notification Logs desktop form from the application

The form includes Submit and Reset controls. Event and registration values associate the message with operational data. Recommended validation includes a valid recipient, nonempty content and a registration belonging to the selected event. Form-builder configuration was not supplied, so Figure 5.2 is an explanatory schematic.

5.3 Reports

Table 5.1: Reports demonstrated

Report	Visible output	Use
All Expense Records	Three rows; Pending and Approved labels	Review expense records and status
All Analytics Summaries	Five event-summary rows	Review attendance and finance values
All Notification Logs	Embedded dashboard report	Inspect communication records

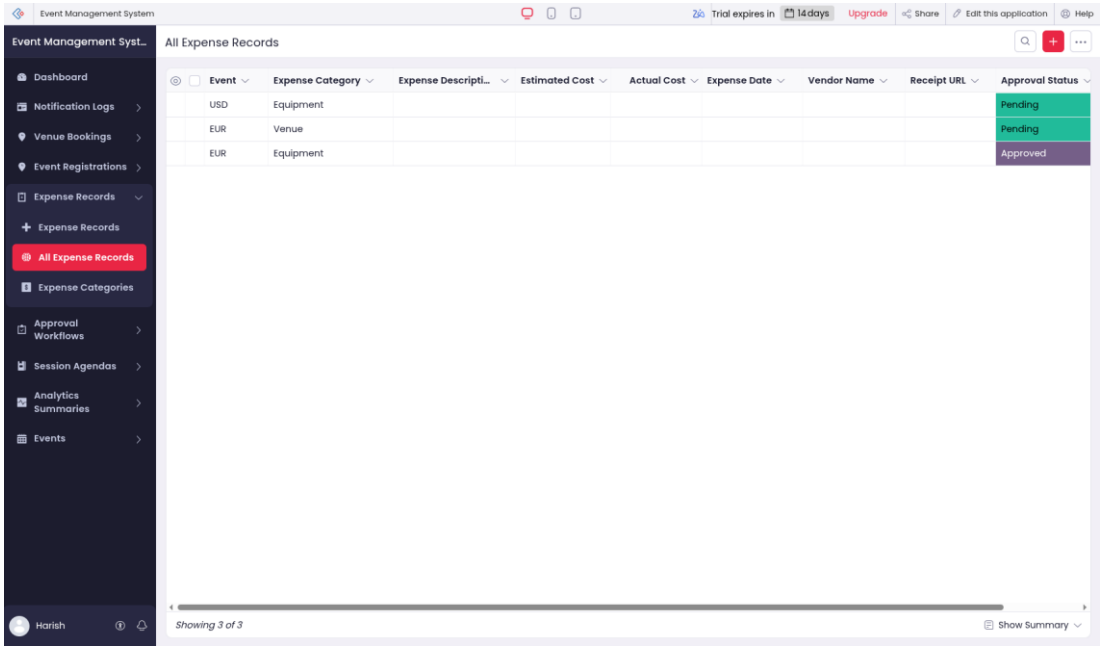


Figure 5.4: All Expense Records report with sample rows

5.4 Pages and Dashboards

The dashboard combines cost cards, revenue and profit plotting, delivery status counters, quick links and an attendee chart. The expense report visibly contains blank cost and supporting-detail cells; its sample status labels alone do not demonstrate completed financial approval.

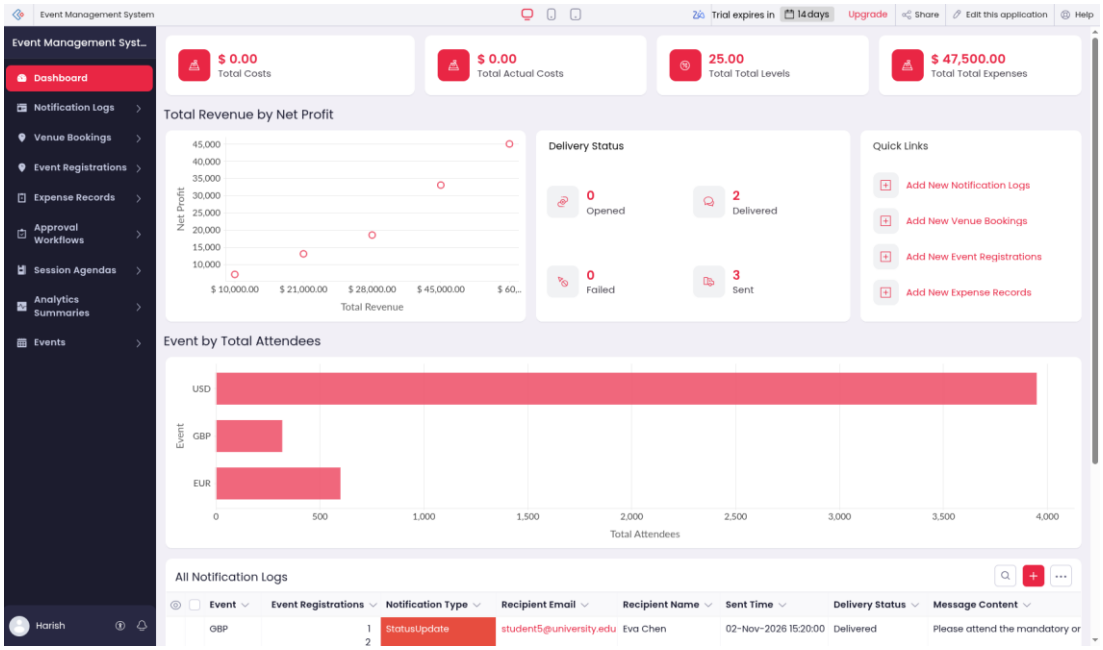


Figure 5.5: Dashboard page viewed on 02 November 2026

5.5 Deluge Scripting

Table 5.2: Proposed Deluge scripts

ID	Proposed rule	Trigger
DS01	Validate event dates and positive capacity	Events On Validate
DS02	Validate expense amounts	Expenses On Validate
DS03	Calculate attendance rate and net profit	Summary On Validate
DS04	Validate event and message content	Notification On Validate

Appendix C provides illustrative scripts with proposed field link names. They are not an export of the running application. Business logic must be tested in the corresponding Creator form before it can be described as implemented.

Code 5.1: Event date validation example

```
if(input.Start_Time == null || input.End_Time == null)
{
    alert "Enter the event start and end time.";
    cancel submit;
}
if(input.End_Time <= input.Start_Time)
{
    alert "End time must be after start time.";
    cancel submit;
}
```

Code 5.2: Attendance-rate calculation example

```
if(input.Total_Registrations > 0)
{
    input.Attendance_Rate =
        (input.Total_Attendees * 100.0 /
         input.Total_Registrations).round(2);
}
else
{
    input.Attendance_Rate = 0;
}
```

5.6 Workflows and Automation

Table 5.3: Proposed workflows

Workflow	Trigger	Intended action
Event validation	Before save	Reject invalid dates or capacity
Registration validation	Before save	Check event, duplicate and capacity
Expense decision	Authorised action	Persist decision, actor and time
Notification dispatch	Approved sending action	Send and record outcome
Summary refresh	After relevant record changes	Recalculate event metrics

5.6 Workflows and Automation Continued

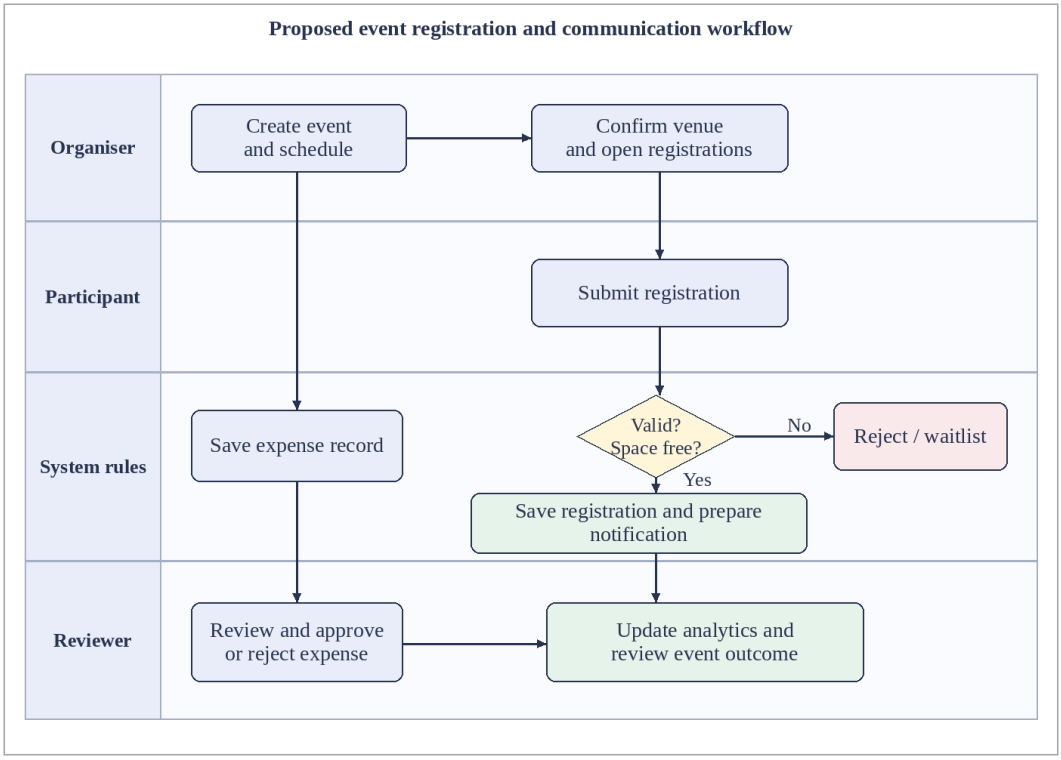


Figure 5.6: Proposed workflow sequence for event operations

The workflow diagram shows the intended dependencies between forms and decisions. A successful form submission must not be interpreted as a successful external notification. Approval and registration rules must be evaluated by the application, including when operations occur through reports.

5.7 Integrations

Table 5.4: Integration scope

Integration	Current evidence	Required next step
Email notification	Log fields and counters only	Configure and verify sending
Calendar or payment service	Not supplied	Evaluate as future integration
REST API / webhook	Not supplied	Define connection and access scope

Code 5.3: Illustrative notification email task

```
// Example only: map fields and test the sender first.
sendmail
[
    from :zoho.adminuserid
    to :input.Recipient_Email
    subject :"Event notification"
    message :input.Message_Content
]
```

The Deluge sendmail task supports event-driven email [4]. The example is not proof of configured sending or delivery. Use a controlled test inbox and an authorised sender. Record dispatch failures separately and avoid assigning Delivered without a delivery outcome.

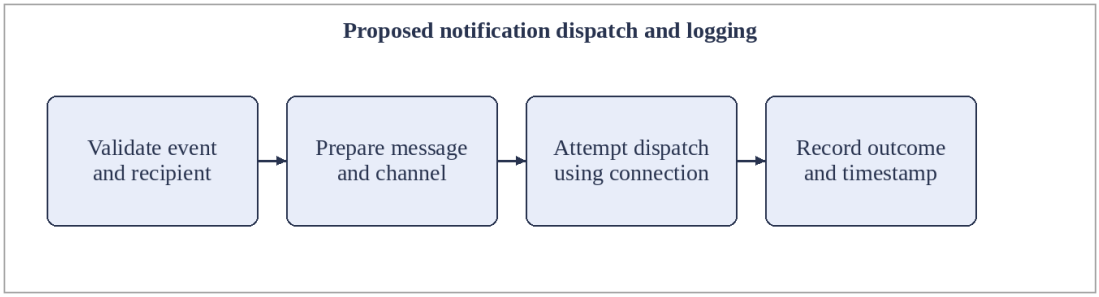


Figure 5.7: Proposed notification dispatch and logging

5.8 User Management and Role Based Access

Recommended roles are Administrator, Organiser, Finance reviewer and Participant. The application owner should configure access by module and record responsibility, then test each role using a separate account. The screenshots show only the Harish session and do not establish role-specific behaviour.

Participant names and email addresses should be accessible only to people who need them for event operations. Expense receipts and exports require their own access review. The permission design in Table 4.5 provides the target configuration.

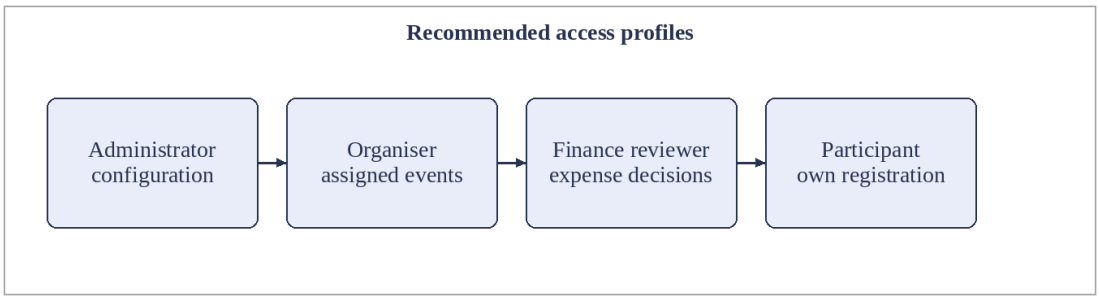


Figure 5.8: Recommended user access profiles

5.9 AI and Analytics Features

The application contains summary reports and dashboard charts. No AI feature is established by the supplied evidence. Analytics calculations and their source-record reconciliation are discussed in Chapter 8.

5.10 Pro Code Extension

No custom widget or external service is evidenced. QR attendance, payment integration or high-concurrency capacity control may require a later extension.

5.11 Mobile and Portal Access

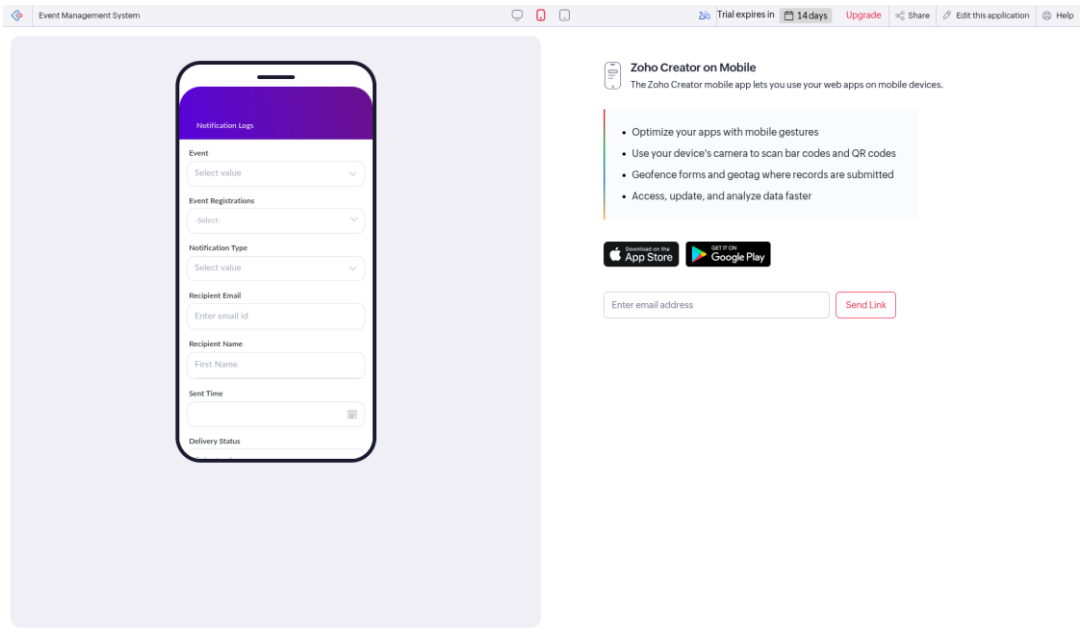


Figure 5.9: Notification Logs mobile layout preview

The screenshot demonstrates a mobile preview, not a physical-device submission or a participant portal. Test scrolling, input controls, validation feedback and permissions on a real phone.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

The report evaluation consists of screenshot inspection and arithmetic checks of visible analytics values. It does not execute application workflows. Functional cases below define the acceptance work still needed; no runtime pass result is invented.

6.2 Test Cases

Table 6.1: Functional test cases

ID	Test action	Expected result	Status
TC01	End time before start	Reject invalid event	Planned
TC02	Duplicate event/email registration	Block unintended duplicate	Planned
TC03	Registration at full capacity	Apply reject or waitlist policy	Planned
TC04	Registration from wrong event	Prevent inconsistent link	Planned
TC05	Overlapping venue bookings	Block confirmed conflict	Planned
TC06	Negative expense amount	Reject invalid cost	Planned
TC07	Unauthorised expense approval	Deny decision action	Planned
TC08	Message to controlled inbox	Verify receipt and recorded result	Planned

Record the tester, date, input record, actual output and evidence when executing a test. A visible status label is not a substitute for observing the rule that creates or changes it.

Table 6.1: Functional test cases continued

ID	Test action	Expected result	Status
TC09	Failed notification and retry	Record failure; prevent duplicate sending	Planned
TC10	Zero-registration summary	Defined rate; no division error	Planned
TC11	Submit from a physical phone	Readable, successful form interaction	Planned
TC12	Access another user's record	Deny unauthorised access	Planned

6.3 Debugging Deluge Scripts

Start with one controlled record and repeatable steps. Compare the input, stored values and expected transition. Inspect the relevant form or report workflow and its execution information before changing the rule. After correction, rerun the failing case and the neighbouring valid case.

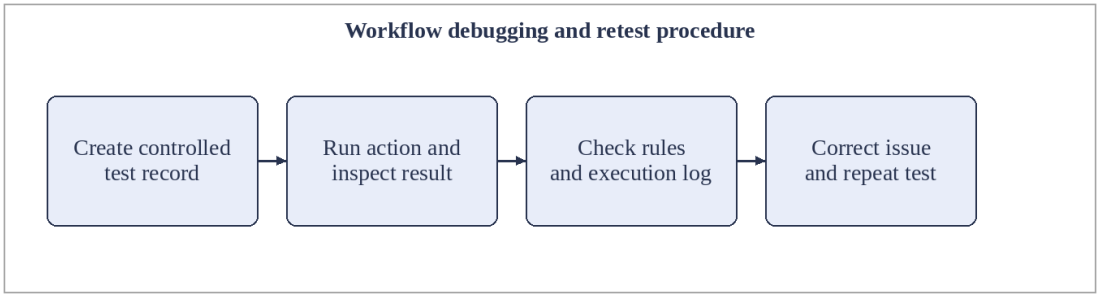


Figure 6.1: Workflow debugging and retest procedure

Static Checks Completed

• All five visible attendance percentages match attendees divided by registrations.

• All five net-profit values match revenue minus expenses.

• The dashboard expense total of 47,500 matches the sum of analytics expense values.

• The expense report shows three rows, and the analytics report shows five rows.

These are screenshot and arithmetic checks. They do not establish automatic calculation, message delivery or role enforcement.

6.4 Defect Log

Table 6.2: Defect and observation log

ID	Observation	Investigation / action	Status
OBS01	Event values show USD, GBP and EUR	Check lookup display fields and imported data	Open
OBS02	Dollar formatting beside currency-like labels	Define currency and aggregation rules	Open
OBS03	Cost details blank in expense rows	Complete source data and reconcile summary	Open
OBS04	Dashboard titles repeat “Total”	Review card labels for clarity	Open
OBS05	Mobile testing limited to preview	Run test on a physical device	Pending

These issues are visible observations or test gaps. A root cause or applied fix is not claimed without inspecting the configuration and reproducing the behaviour.

6.5 User Acceptance Testing

Table 6.3: User acceptance plan

User	Task	Acceptance evidence
Organiser	Maintain an event and linked records	Completed tasks and recorded feedback
Reviewer	Approve or reject an expense	Correct permissions and decision history
Participant	Submit and view own registration	Successful flow without other-user access
Administrator	Review logs, exports and access	Documented checks and release decision

No user acceptance sign-off was supplied. The table defines the users and evidence required for final acceptance.

CHAPTER 7
DEPLOYMENT SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The screenshots establish browser access and a mobile preview, but do not identify a development, stage or production environment. Figure 7.1 presents a recommended release sequence, not an environment-settings screenshot.

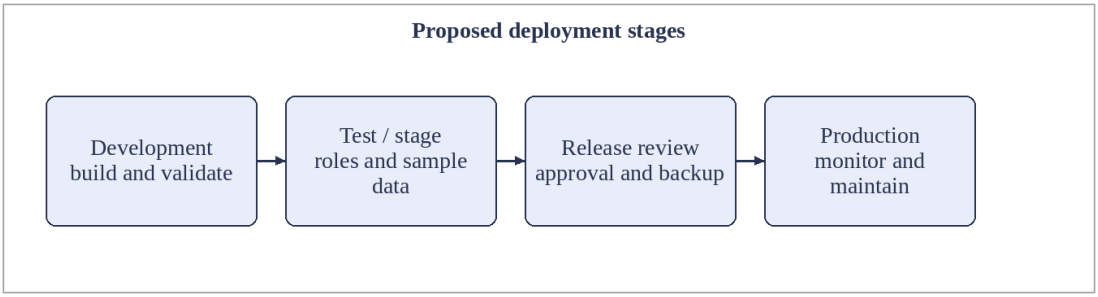


Figure 7.1: Proposed deployment stages

Table 7.1: Deployment checklist

Area	Required verification
Data	Correct event labels; complete sample and source records
Rules	Execute validation, approval and notification tests
Access	Test independent accounts and direct record links
Backup	Verify export and restore procedure
Release	Record environment, version and approval
Mobile	Complete a physical-device acceptance test

7.2 Performance Optimisation

Small screenshot samples do not establish scalability. Measure dashboard loading and record operations at representative event volumes. Filter reports by event and date and avoid unnecessary full-dataset recalculation when measurements show it is needed.

Table 7.2: Performance evaluation plan

Component	Measurement	Current result
Dashboard	Load time at defined record counts	Not measured
Reports	Open, filter and search time	Not measured
Workflow	Validation and save duration	Not measured

7.3 Security Features

Table 7.3: Recommended security controls

Control	Purpose
Role permissions	Restrict modules, exports and record operations
Record ownership	Limit users to assigned events or own registration
Personal data handling	Minimise exposure of participant details
Receipt access	Prevent unauthorised access through shared links

7.4 Governance and Administration

Retain the actor, action, target record, before-and-after state and timestamp for important decisions. Document workflow changes and summary formulas. Define retention and backup responsibilities before operational use.

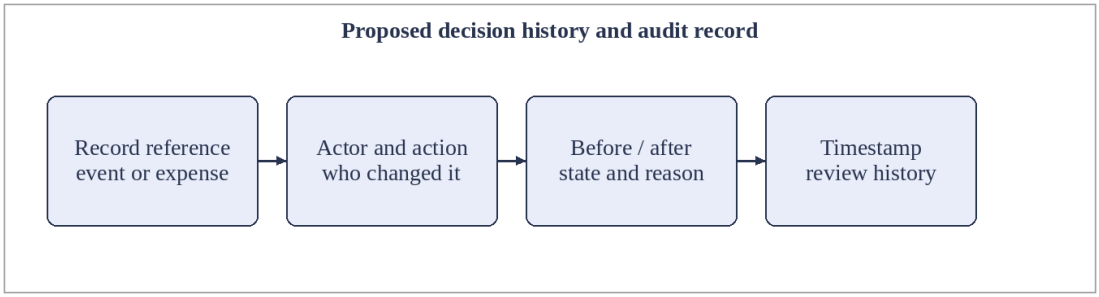


Figure 7.2: Proposed audit record structure

The supplied evidence does not establish a configured audit trail or tested backup. The diagram identifies the information needed for traceable event and expense decisions.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The prototype brings event operations into one navigation structure. The supplied notification form captures an event reference, registration, recipient, message and communication channel. The expense report exposes status information, and the analytics report provides attendance and financial summaries.

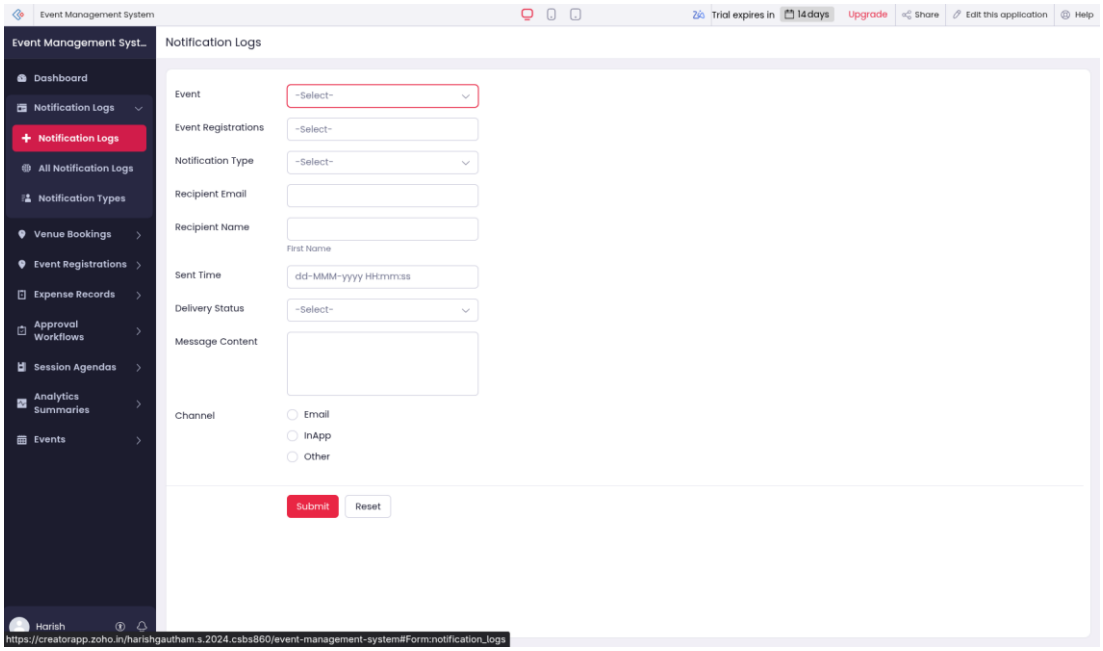


Figure 8.1: Notification entry interface from the supplied application

The sample analytics contain 5,950 registrations and 4,870 attendees in total. The overall attendance rate is 4,870 / 5,950 × 100 = 81.85%. Net-profit arithmetic is consistent for all five rows. The waitlist counts sum to 600.

The Event column shows currency-code-like labels while amounts use dollar formatting. Financial totals therefore serve as arithmetic checks only until event identity and a common currency are confirmed. The visible source expense rows also require reconciliation with their summary values.

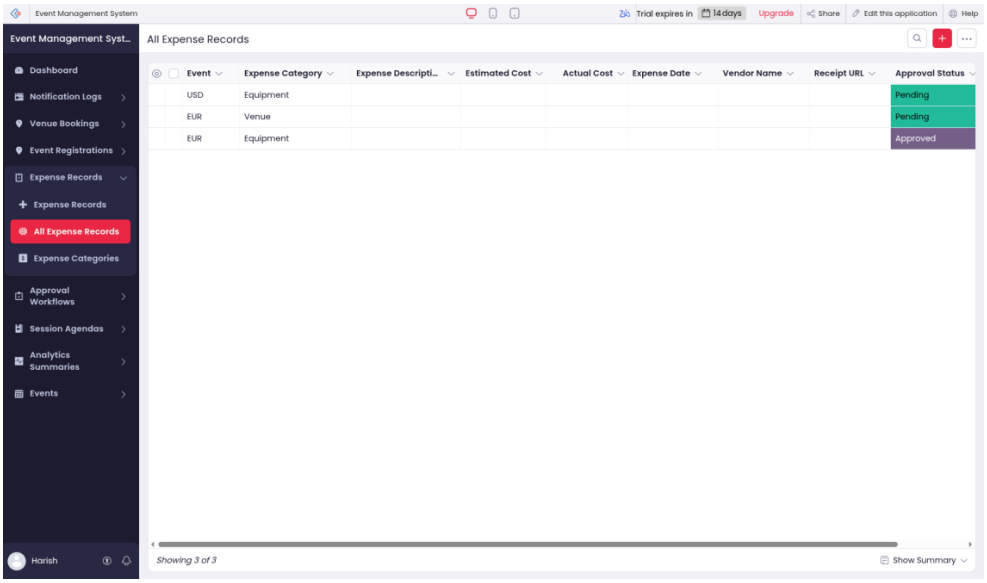


Figure 8.2: Expense approval status report

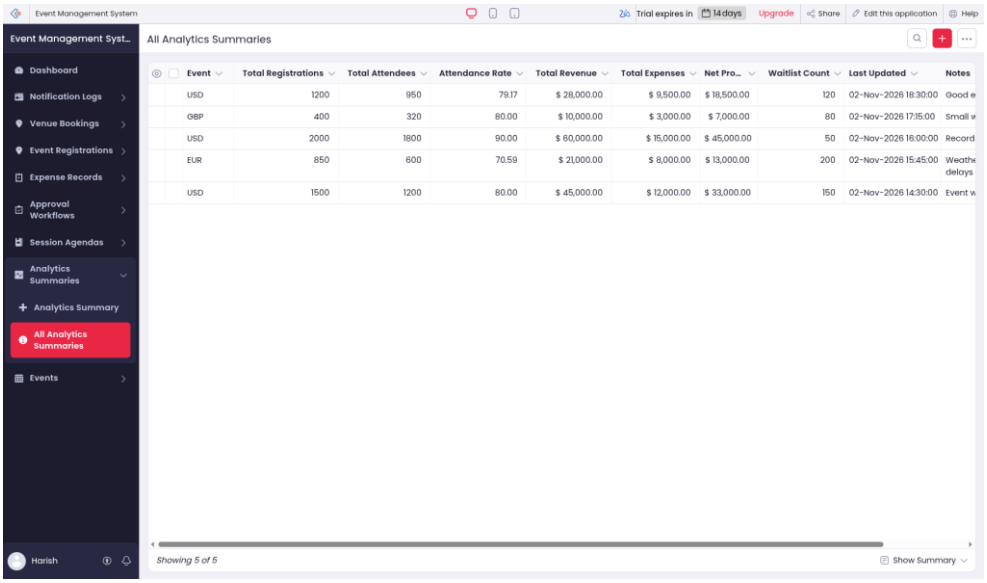


Figure 8.3: Event analytics summary report

8.2 Achievement of Objectives

Table 8.1: Mapping of objectives to outcomes

Objective	**Outcome**
Common event workspace	Eight modules and dashboard visible
Notification and expense records	Form and status reports demonstrated
Analytics and mobile layout	Charts, summary rows and preview demonstrated
Automation and secure operation	Requires runtime and independent-role tests

8.3 Comparison with the Existing System

Table 8.2: Existing process versus proposed system

Aspect	Manual baseline	Proposed system
Information	Separate files and messages	Related event records
Approvals	Dispersed decisions	Structured status and history
Communication	Manual messages	Linked logs and planned dispatch
Reporting	Manual consolidation	Reports and event summaries
Access	Shared files	Role and record permissions

The intended benefit is less reconciliation work and clearer operational visibility. No timed comparison or user survey was supplied, so no percentage improvement is claimed.

8.4 Limitations of the Low Code Approach

Visual builders simplify interface construction but do not determine correct field mappings, financial interpretation or permissions. Platform-specific configuration affects portability. Capacity checks under concurrent submissions and specialised integrations require additional design and testing.

8.5 No Code Low Code and Pro Code Spectrum

Forms, navigation and report layouts represent visual configuration. Deluge validation, calculations and workflows form the low-code layer. A custom API, widget or external service would be a pro-code extension. No such extension is established by the supplied build.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Smart Event Management System demonstrates how a Zoho Creator application can organise event administration within a common interface. The supplied screens show a dashboard, notification data capture, expense status reporting, analytics summaries and a mobile form preview. The displayed analytics percentages and profit values are arithmetically consistent with the visible sample data.

The prototype provides a useful foundation for event coordination. Completing the system requires clearer event identifiers, defined currency handling, reconciliation between operational and summary records, and executed tests for workflows, access control and notification delivery. These are specific development tasks arising from the demonstrated application.

9.2 Future Enhancements

• Replace currency-code-like Event labels with meaningful event names and use a separate currency field.

• Recalculate analytics from validated source records and show when each summary was refreshed.

• Implement and test duplicate-registration, capacity and venue-conflict rules.

• Configure approval routing with reviewer identity, decision timestamps and reasons.

• Verify notification sending, reminders, delivery outcomes and safe retry handling.

• Add QR-based attendance, participant self-service and calendar integration after the core workflow is stable.

• Complete independent role testing, physical-device testing and a documented deployment review.

9.3 Learning Outcomes

The project illustrates the relationship between forms, data design, reports and business rules. Reviewing the outputs also shows why a visible dashboard must be supported by meaningful labels and reliable source data. These skills apply to business analysis, low-code development, application testing and operational reporting.

REFERENCES

[1] Zoho Corporation. “Understand Application.” Zoho Creator Help.

https://help.zoho.com/portal/en/kb/creator/developer-guide/creating-applications/understand-applications/articles/understand-application

[2] Zoho Corporation. “Form Workflows.” Zoho Creator Knowledge Base.

https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/create-and-manage-form-workflows

[3] Zoho Corporation. “Understanding permissions.” Zoho Creator Help.

https://help.zoho.com/portal/en/kb/creator/developer-guide/application-settings/manage-and-configure-user-permissions/articles/understand-permissions

[4] Zoho Corporation. “Send Email using Deluge.” Deluge Help.

https://www.zoho.com/deluge/help/send-email-using-deluge.html

Online sources consulted on 02 November 2026.

Project Evidence and Formatting Reference

[5] Harish Gautham S. Five Event Management System screenshots captured on 02 November 2026 at 19:34:11, 19:34:21, 19:34:37, 19:34:43 and 19:35:35. Used in Figures 5.1, 5.3, 5.4, 5.5, 5.9 and 8.1 to 8.3 and as the basis for the screenshot review.

[6] Hariharan N. GateFlow - Smart Visitor Management System. Supplied project report, file 241401024_LowCode_Project_Report.pdf, November 2026. Used as a reference for the academic report structure, course details and institutional presentation. Its application content and source code are not reused as this project’s implementation.

The application figures reproduce the supplied screenshots without changing interface values. The proposed data model, test plan and illustrative scripts were prepared specifically for this event management report.

APPENDIX A
ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature coverage checklist

Feature	Evidence status	Report location
Application and menu	Observed	Figure 5.1
Desktop form	Observed	Figure 5.3
Selection, text, date and multiline fields	Observed	Section 4.3
Related-record selectors	Controls observed; relationship settings not verified	Figure 5.3
List reports	Observed	Figures 5.4 and 8.3
Dashboard and charts	Observed	Figure 5.5
Status formatting	Observed	Figure 8.2
Form validation	Proposed; not executed	Appendix C
Deluge functions	No exported application source	Section 5.5
Form workflows	Design described; configuration not verified	Figure 5.6
Notifications	Log interface observed; delivery not verified	Section 5.7
Approval process	Module and statuses visible; routing not verified	Section 5.6

The checklist distinguishes interface evidence from configured behaviour. A visual control or menu entry is not sufficient to establish a validated workflow.

Table A.1: Feature coverage checklist continued

Feature	Evidence status	Report location
Scheduled workflows	Not evidenced	Section 5.6
Blueprint / batch workflows	Not evidenced	Section 5.6
REST API or webhook	Not evidenced	Section 5.7
Roles and permission profiles	Proposed; independent accounts not tested	Table 4.5
AI-assisted application feature	Not evidenced	Section 5.9
Mobile access	Preview observed; real-device test pending	Figure 5.9
External customer portal	Not evidenced	Section 5.11
Performance measurement	Not performed	Table 7.2
Production deployment	Not established	Section 7.1
Audit trail and backup	Recommended; not verified	Section 7.4
Pro-code extension	Not evidenced	Section 8.5

Evidence Completion Plan

To complete the project assessment, capture the field relationships in the builder, save and test the validation rules, demonstrate an authorised expense decision, verify a notification in a controlled inbox and test access with separate accounts. Record release and backup checks before operational deployment.

No combined “implemented feature” score is assigned because the screenshots alone cannot determine the configuration of every platform feature.

APPENDIX B
COURSE OUTCOME MAPPING

Table B.1 Course outcome mapping

CO	Outcome focus	Report evidence
CO1	Explain low-code principles, architecture and business value	Chapters 1 and 2 explain the problem and platform approach
CO2	Develop applications using forms, reports, workflows and Deluge	Chapters 4 and 5 document design and screens; Appendix C supplies illustrative rules
CO3	Understand integrations and role-based security	Sections 4.6 and 5.7 describe the access model and integration scope
CO4	Deploy, secure and manage applications	Chapter 7 presents deployment and governance requirements
CO5	Evaluate low-code strategies and the transition to pro-code	Sections 8.4 and 8.5 discuss limitations and extension choices

Assessment Notes

The mapping follows the course-outcome themes in the reference report. It identifies where a topic is addressed; it is not an examiner's assessment or proof that every outcome has been demonstrated through an executed application task.

The strongest supplied implementation evidence concerns forms, reports, dashboard presentation and mobile preview. Workflow execution, integration, separate user permissions and production administration need additional configuration or test evidence for a complete practical assessment.

A demonstration can begin with the dashboard, show the notification form, explain the expense status report, and verify the analytics calculations. The presenter should then explain the remaining validation and deployment tasks using the test plan and feature checklist.

APPENDIX C
ILLUSTRATIVE DELUGE SOURCE CODE

The examples below are prepared for the proposed event management schema. They are not exported application source. Confirm all form and field link names, configure mandatory fields, save in the correct workflow and run the tests before deployment.

C.1 Event Date and Capacity Validation

Proposed Events form, On Validate. Required fields: Start_Time, End_Time and Capacity.

```
if(input.Start_Time == null || input.End_Time == null)
{
    alert "Enter the event start and end time.";
    cancel submit;
}
if(input.End_Time <= input.Start_Time)
{
    alert "End time must be after start time.";
    cancel submit;
}
if(input.Capacity == null || input.Capacity <= 0)
{
    alert "Capacity must be greater than zero.";
    cancel submit;
}
```

C.2 Expense Amount Validation

Proposed Expense Records form, On Validate. Make the relevant fields mandatory separately when required by the process.

```
if(input.Estimated_Cost != null &&
   input.Estimated_Cost < 0)
{
    alert "Estimated cost cannot be negative.";
    cancel submit;
}
if(input.Actual_Cost != null && input.Actual_Cost < 0)
{
    alert "Actual cost cannot be negative.";
    cancel submit;
}
```

C.3 Analytics Summary Calculation

Proposed Analytics Summaries form, On Validate. Counts and monetary inputs should be required and nonnegative. This example calculates from supplied summary inputs; it does not aggregate operational records.

```
if(input.Total_Attendees > input.Total_Registrations)
{
    alert "Attendees cannot exceed registrations.";
    cancel submit;
}
if(input.Total_Registrations > 0)
{
    input.Attendance_Rate =
        (input.Total_Attendees * 100.0 /
         input.Total_Registrations).round(2);
}
else
{
    input.Attendance_Rate = 0;
}
input.Net_Profit = input.Total_Revenue - input.Total_Expenses;
```

C.3.1 Input Requirements

Make Total_Registrations, Total_Attendees, Total_Revenue and Total_Expenses mandatory numeric inputs. Reject negative values before calculating. Define whether revenue includes only confirmed payments and whether expenses include only approved actual costs. Use the same event and currency for both amounts.

```
if(input.Total_Registrations < 0 ||
   input.Total_Attendees < 0 ||
   input.Total_Revenue < 0 || input.Total_Expenses < 0)
{
    alert "Counts and amounts cannot be negative.";
    cancel submit;
}
```

The example computes from summary inputs. Automatic aggregation from registrations and expenses requires additional queries and must be reconciled with source records. It should not silently mix different currencies.

C.4 Notification Content Validation

Proposed Notification Logs form, On Validate. Use an Email field for address validation and configure required fields in the form builder.

```
if(input.Event == null)
{
    alert "Select an event.";
    cancel submit;
}
if(input.Message_Content == null ||
   input.Message_Content.trim() == "")
{
    alert "Enter the notification message.";
    cancel submit;
}
```

C.4.1 Dispatch Example

After validation, a separate authorised workflow may attempt email dispatch. Select a valid sender and controlled test recipient. A saved log entry must remain distinct from a confirmed delivery outcome [4].

```
// Illustrative dispatch task; not installed source.
sendmail
[
    from :zoho.adminuserid
    to :input.Recipient_Email
    subject :"Event notification"
    message :input.Message_Content
]
```

C.4.2 Notification State Rules

• Set Sent Time when a real dispatch attempt occurs, not when an empty form opens.

• Preserve failure information and allow only controlled retries.

• Do not change status to Delivered without a supported delivery result.

• Validate that any selected registration belongs to the selected event.

C.5 Registration and Approval Algorithms

The following pseudocode specifies rules that require the actual schema, permission model and concurrency behaviour. It is not executable Deluge.

```
REGISTRATION VALIDATION
  require a valid, open event
  normalise the participant email
  reject an existing registration for this event/email
  count registrations that occupy capacity
  if capacity is full:
      reject or create a waitlist record by policy
  otherwise:
      save the registration and its event reference

EXPENSE DECISION
  load the selected expense
  verify the actor has permission for this event
  require the current state to be Pending
  validate amount, category and required receipt
  store Approve or Reject with reason
  record the actor, timestamp and previous state
  refresh the relevant event summary
```

C.6 Verification Before Deployment

Test both valid and invalid submissions. Check null inputs, zero registrations, duplicate records, full-capacity events and unauthorised decisions. Repeat capacity and booking tests with simultaneous requests; a simple count followed by a save does not by itself establish concurrency safety.

Keep a versioned copy of each deployed rule, record its trigger and identify the reports that depend on it. Preserve earlier decision history when correcting records. Confirm that the application source matches the documentation after every change.

APPENDIX D
USER MANUAL

D.1 Accessing the Application

Sign in to Zoho Creator using an account authorised for the application and open Event Management System. The owner session in the supplied screenshots is labelled Harish. The access route is recorded in Project at a Glance. This report does not provide credentials or establish public access.

D.2 Reviewing the Dashboard

Open Dashboard in the left menu. Review the financial cards, revenue-versus-profit chart, delivery status counters and attendee chart. Use the quick links to open the relevant data-entry module. Interpret event labels and financial totals using the data-quality notes in Chapter 8.

D.3 Entering a Notification Log

Expand Notification Logs and select the Notification Logs form. Choose the event and related registration, select the notification type, and enter the recipient email and name. Provide the timestamp and status according to the application's configured process, enter the message, select a channel and choose Submit. Use Reset to clear the form. Saving a log is not confirmation that a message was delivered.

D.4 Reviewing Expenses and Analytics

Expand Expense Records and open All Expense Records to inspect categories and approval states. Review the cost, vendor and receipt details before using the record for a decision. Open Analytics Summaries and select All Analytics Summaries to review registrations, attendees, rates, revenue, expenses and net profit. Check Last Updated before relying on a summary.

D.5 Checking the Mobile Layout

Use the mobile preview control to inspect the notification form. Scroll to the lower fields and check that labels and controls remain readable. Complete a separate test on a real phone before confirming mobile operational readiness.

D.6 Reporting an Issue

Record the affected module, event or record reference, action attempted, expected result and actual message. Include a screenshot that avoids unnecessary personal data. Ask the application owner to review the issue and repeat the failed test after correction.